

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1
по курсу “Логические основы интеллектуальных систем”
Вариант 2

Выполнил:
Студент гр. 421701

Аникеев А.Д

Проверил:

Ивашенко В. П.

Минск
2026

Тема:

Представление и синтаксическая проверка формул языка логики высказываний.

Цель:

Приобрести навыки программирования алгоритмов синтаксического разбора формул языка логики высказываний.

Задание:

Проверить, является ли формула сокращенного языка логики высказываний общезначимой (тавтологией).

Грамматика языка логики высказываний:

<константа>	::=1 0
<символ>	::=A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
<отрицание>	::= !
<конъюнкция>	::= ∧
<дизъюнкция>	::= ∨
<импликация>	::= ->
<эквиваленция>	::= ~
<открывающая скобка>	::= (
<закрывающая скобка>	::=)
<бинарная связка>	::= <конъюнкция> <дизъюнкция> <импликация> <эквиваленция>
<атомарная формула>	::= <латинская заглавная буква>
<унарная сложная формула>	::= <открывающая скобка><отрицание><формула><закрывающая скобка>
<бинарная сложная формула>	::= <открывающая скобка><формула><бинарная связка><формула><закрывающая скобка>
<формула>	::=<логическая константа> <атомарная формула> <сложная формула>

Общезначимая формула (или тавтология) - это формула, которая истинна на всех её возможных интерпретациях

Блок-схемы функций программы:

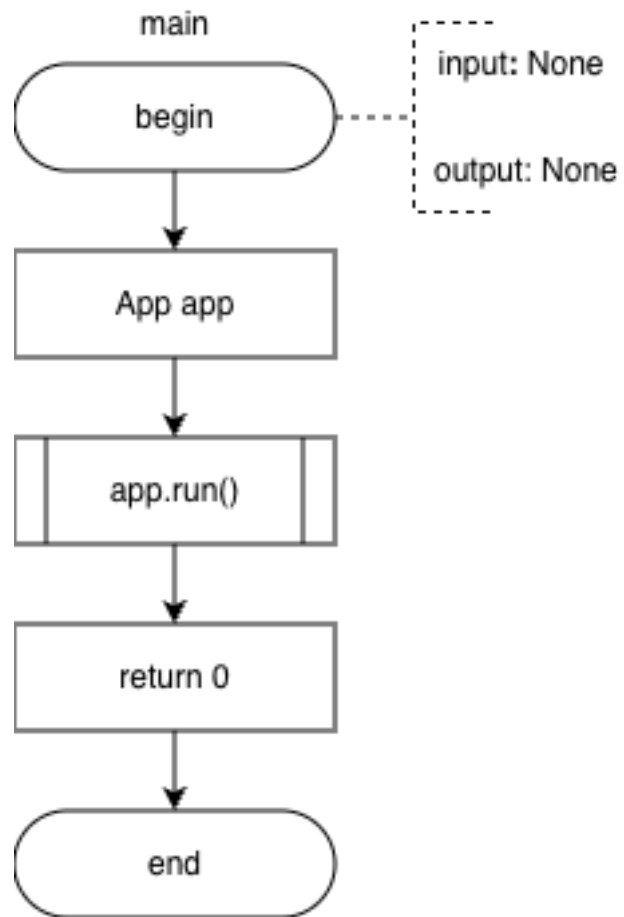


Рис 1. Функция `main()`.

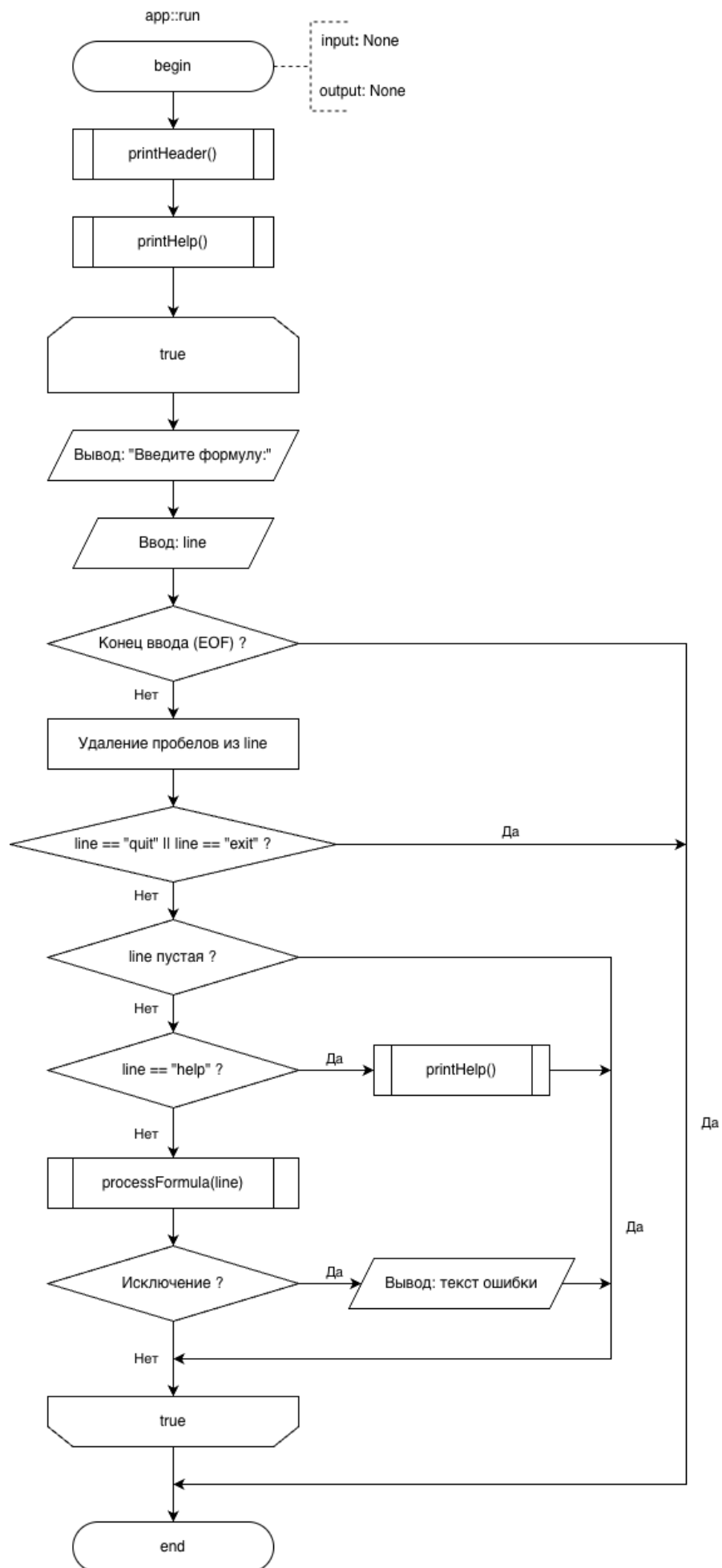


Рис 2.1. Метод run() класса App.

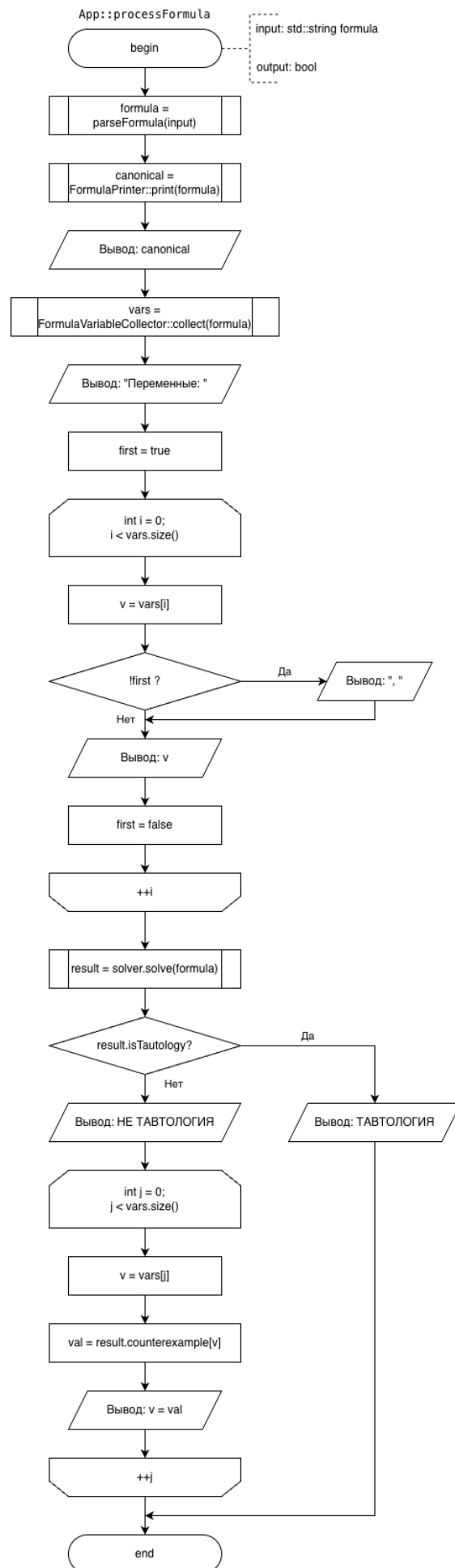


Рис 2.2. Метод processFormula() класса App.

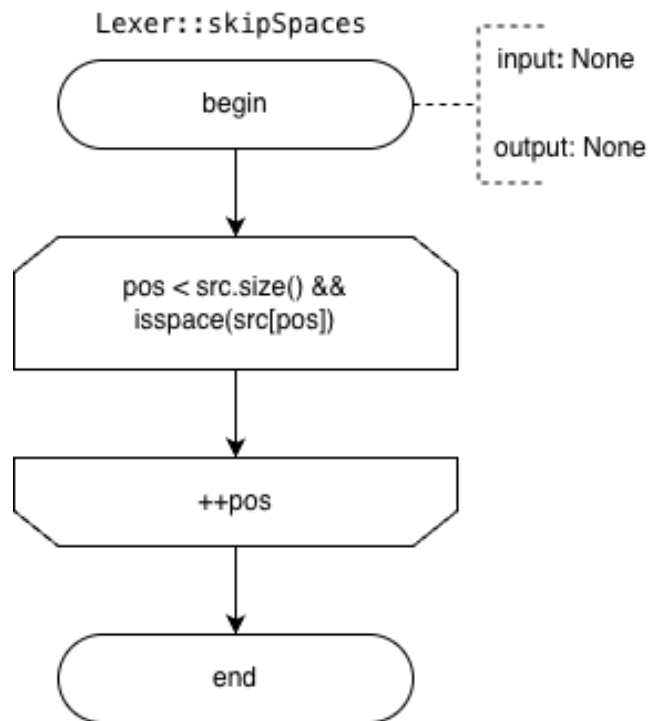


Рис 3.1. Метод skipSpaces() класса Lexer.

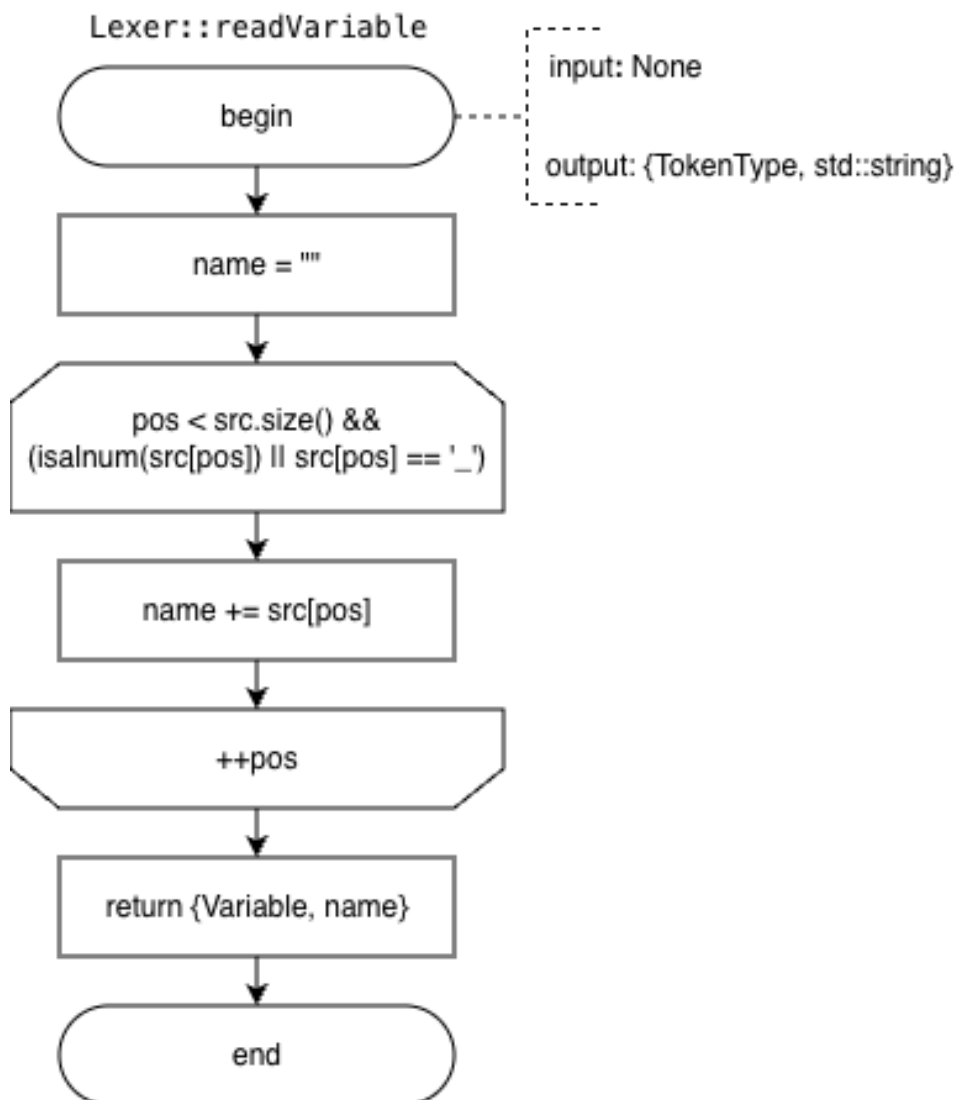


Рис 3.2. Метод readVariable() класса Lexer.

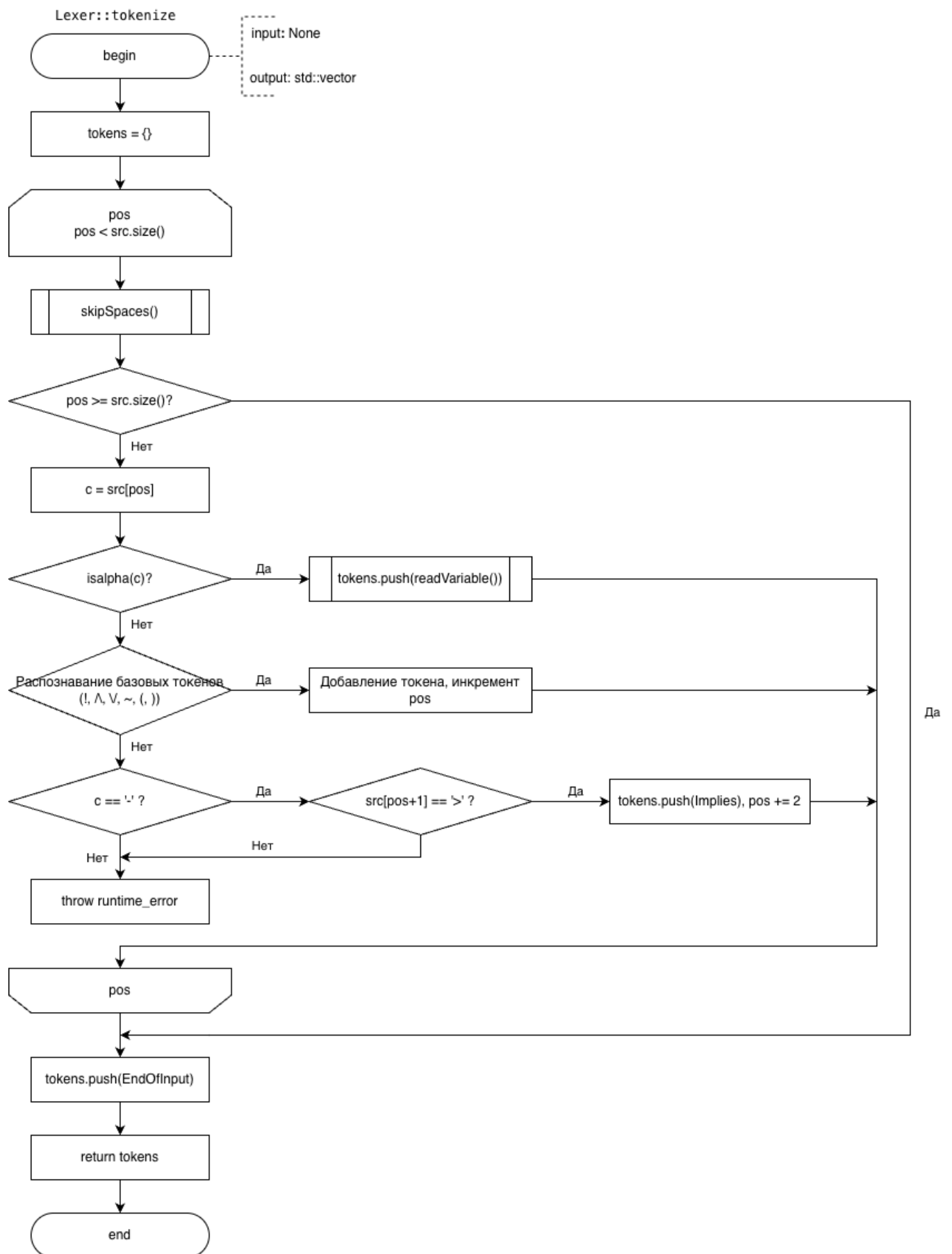


Рис 3.3. Метод tokenize() класса Lexer.

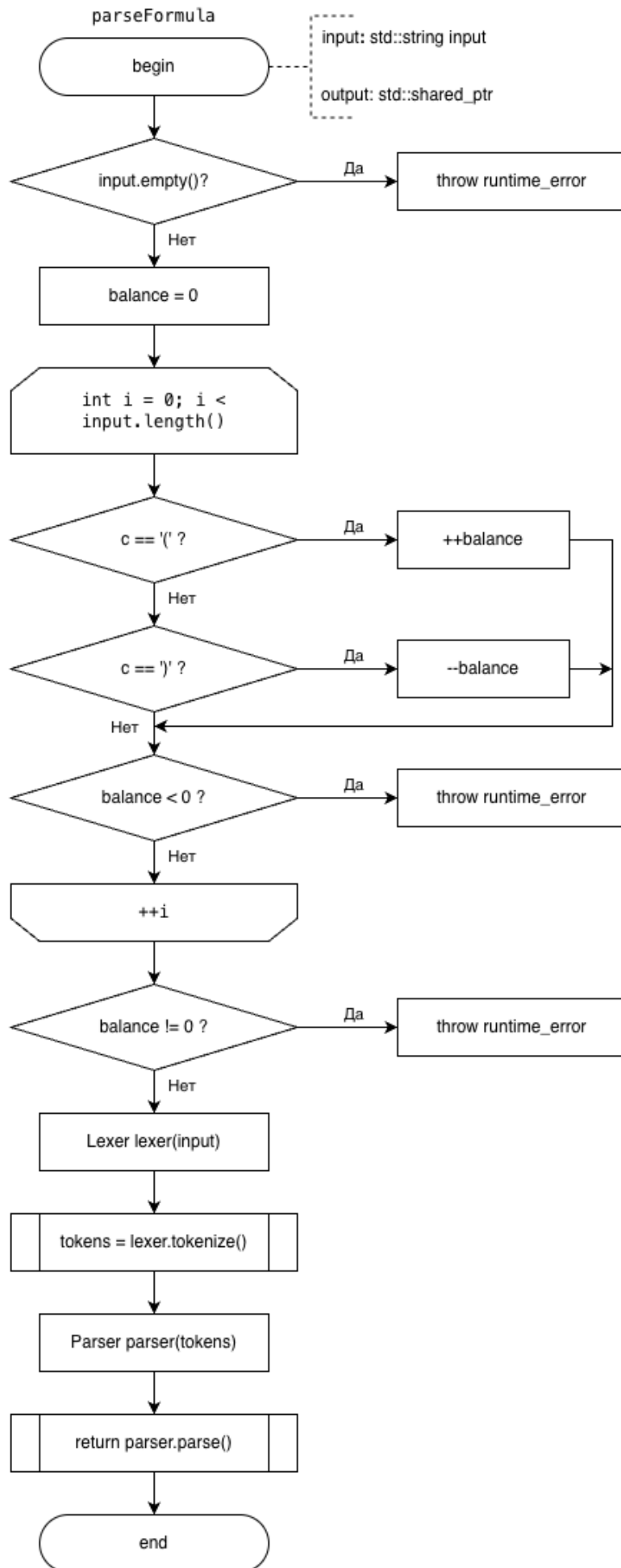


Рис 4. Функция `parseFormula()`.

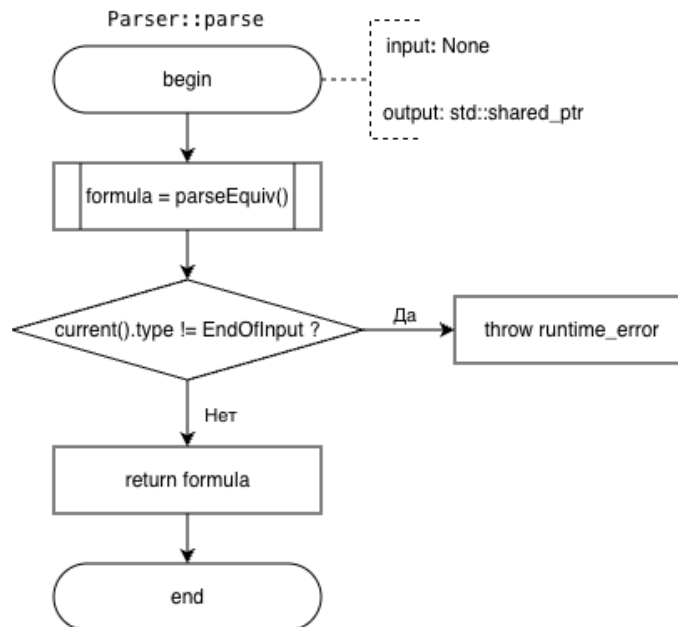


Рис 5.1. Метод parse() класса Parser.

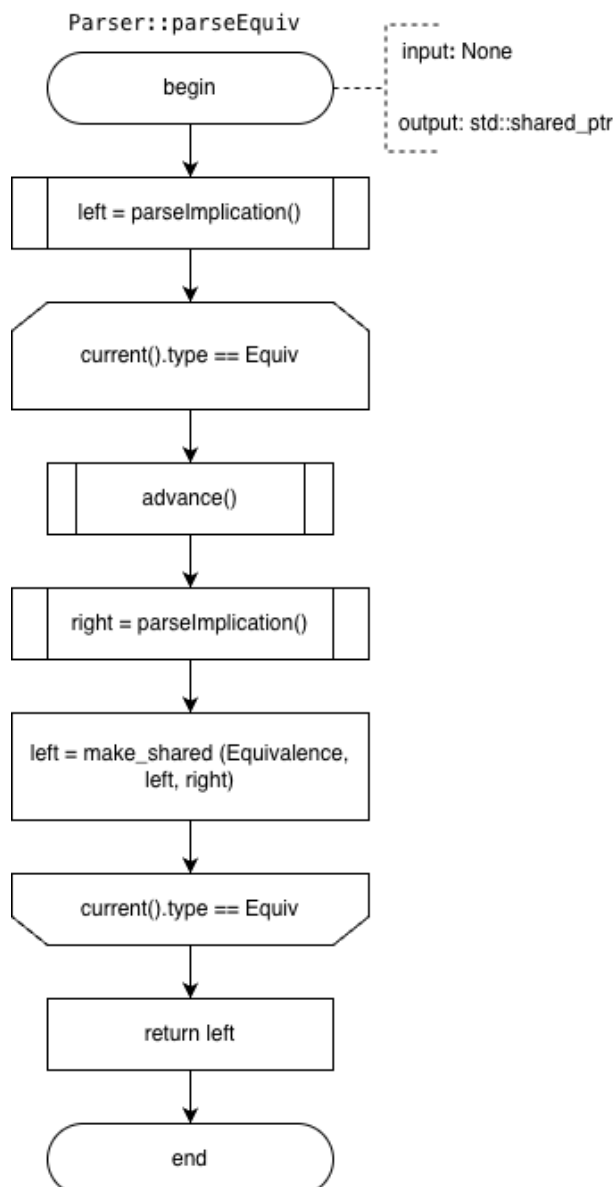


Рис 5.2. Метод parseEquiv() класса Parser.

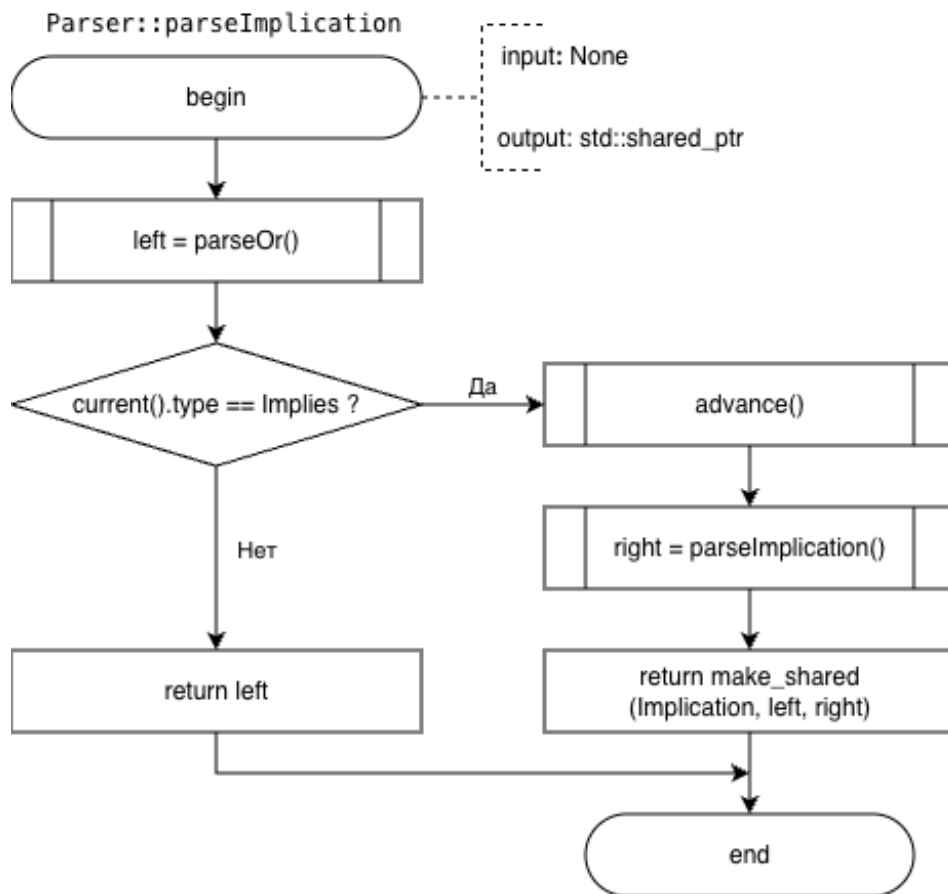


Рис 5.3. Метод parseImplication() класса Parser.

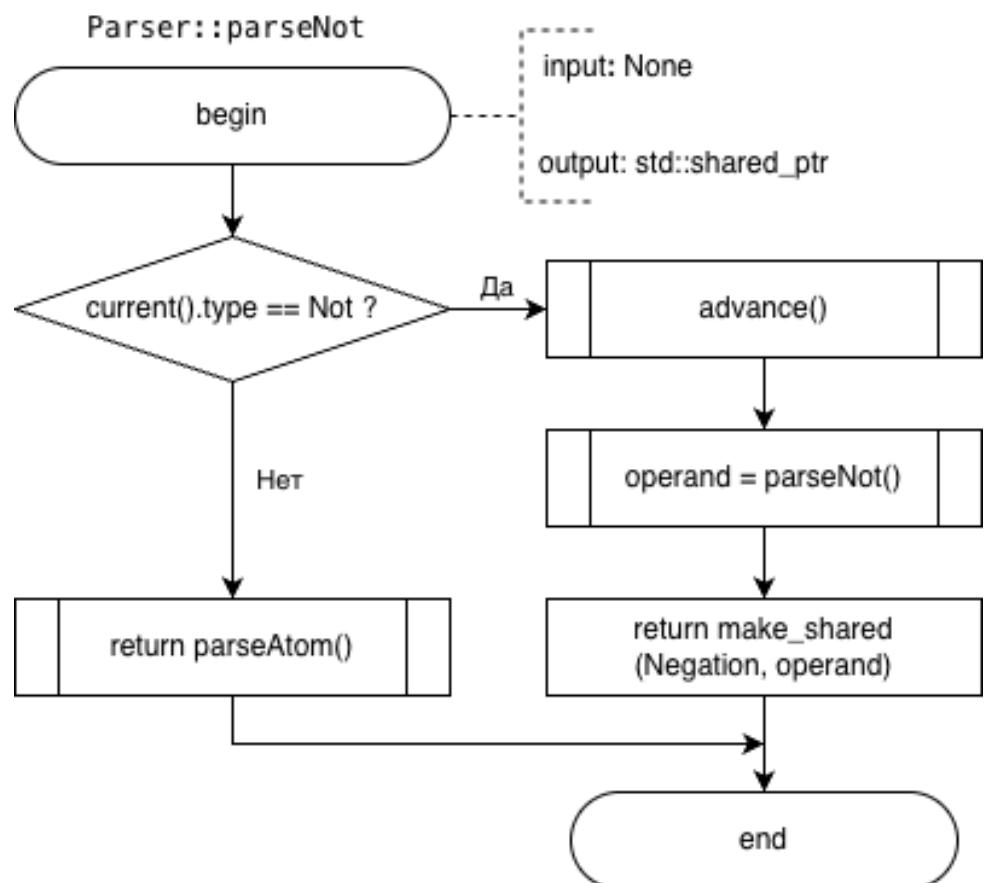


Рис 5.4. Метод parseNot() класса Parser.

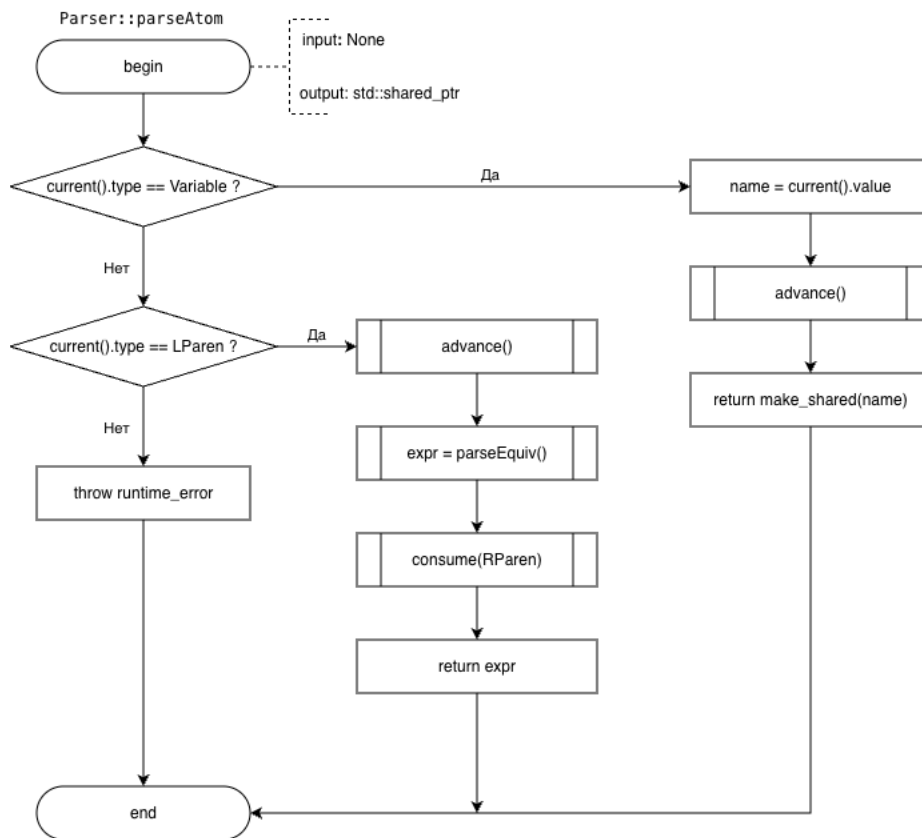


Рис 5.5. Метод parseAtom() класса Parser.

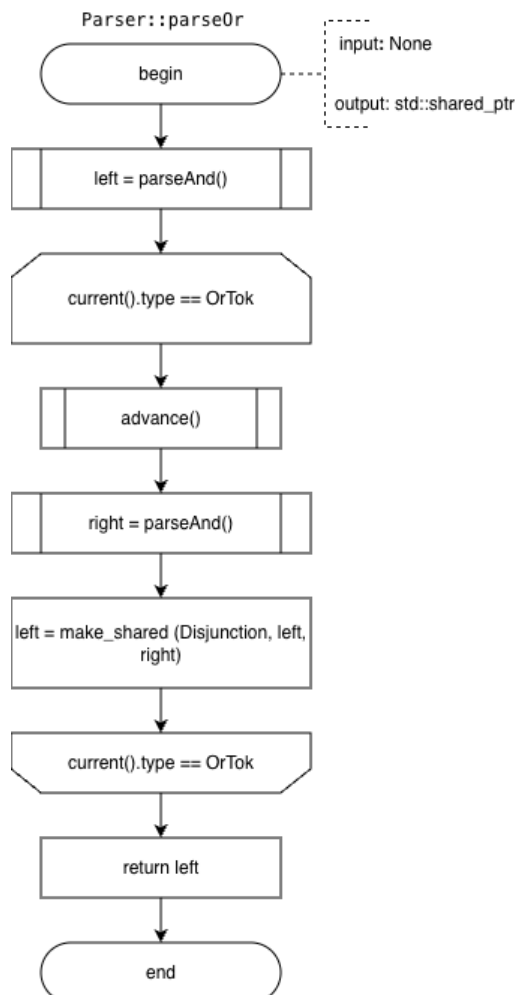


Рис 5.6. Метод parseOr() класса Parser.

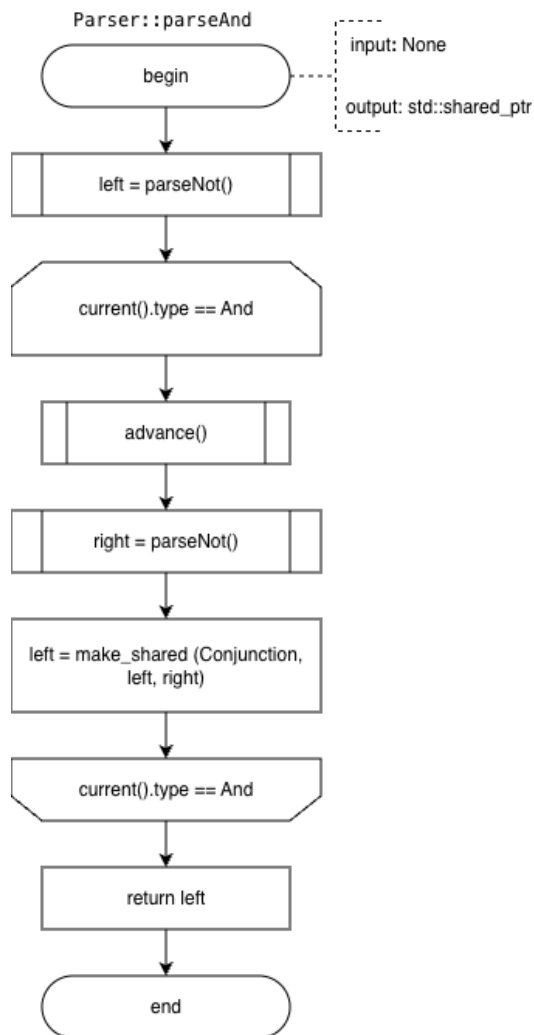


Рис 5.7. Метод parseAnd() класса Parser.

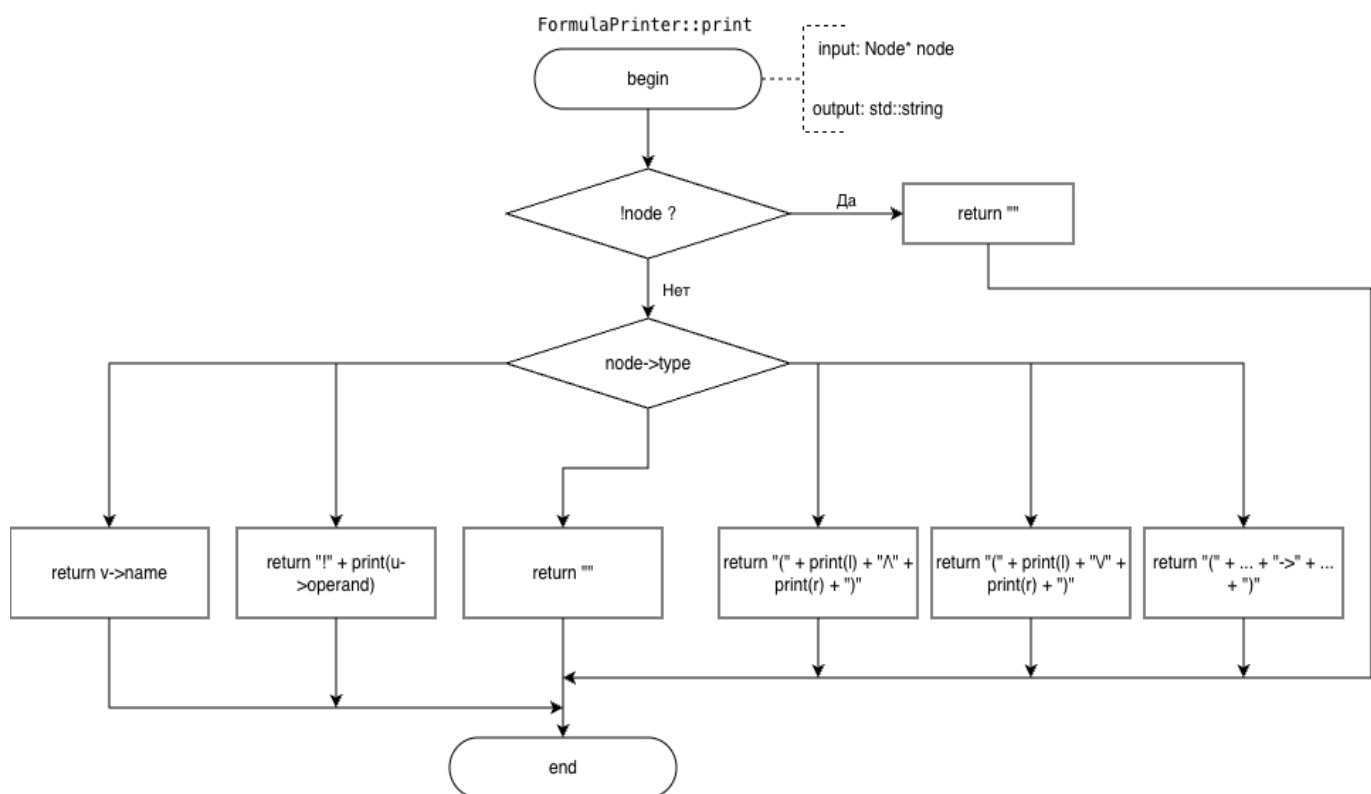


Рис 6. Метод print() класса FormulaPrinter.

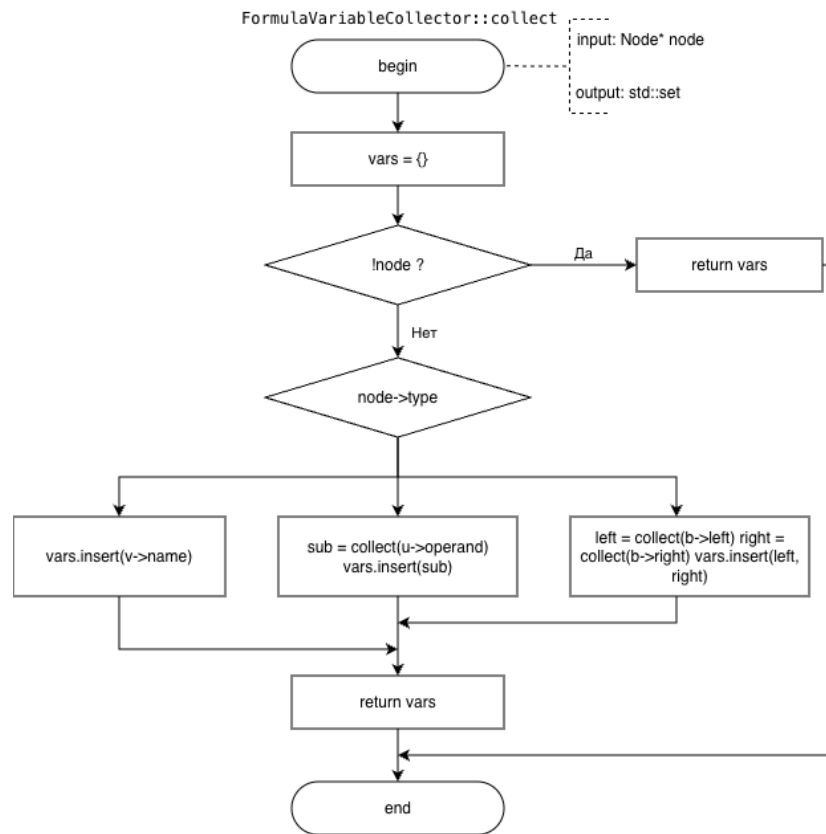


Рис 7. Метод collect() класса FormulaVariableCollector.

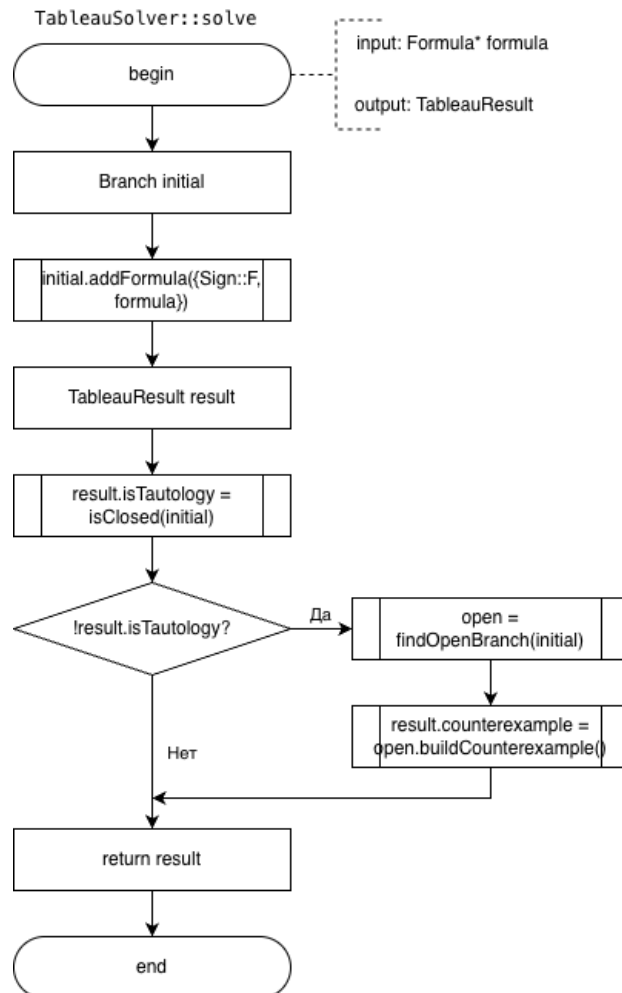


Рис 8.1. Метод solve() класса TableauSolver.

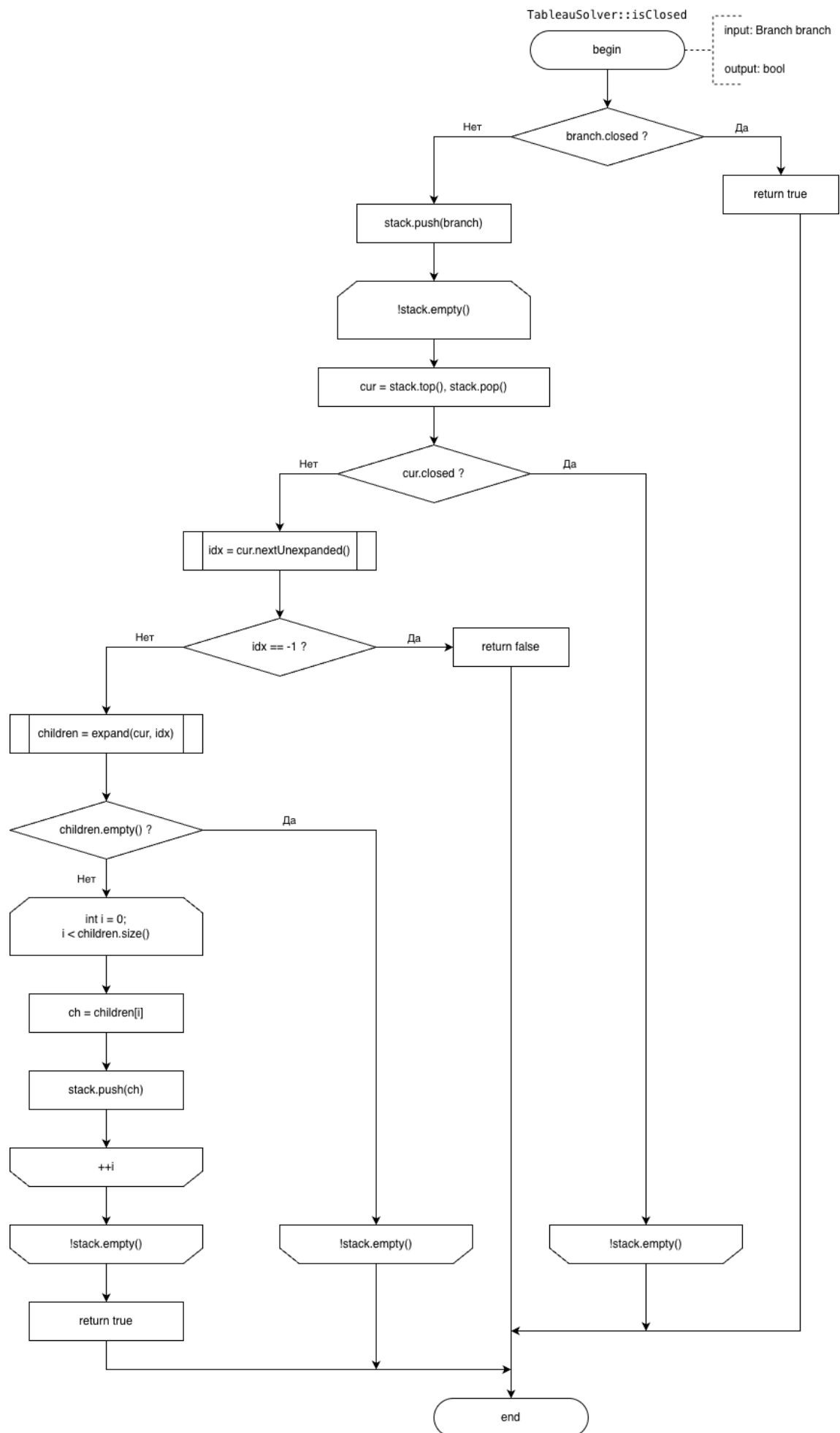


Рис 8.2. Метод `isClosed()` класса `TableauSolver`.

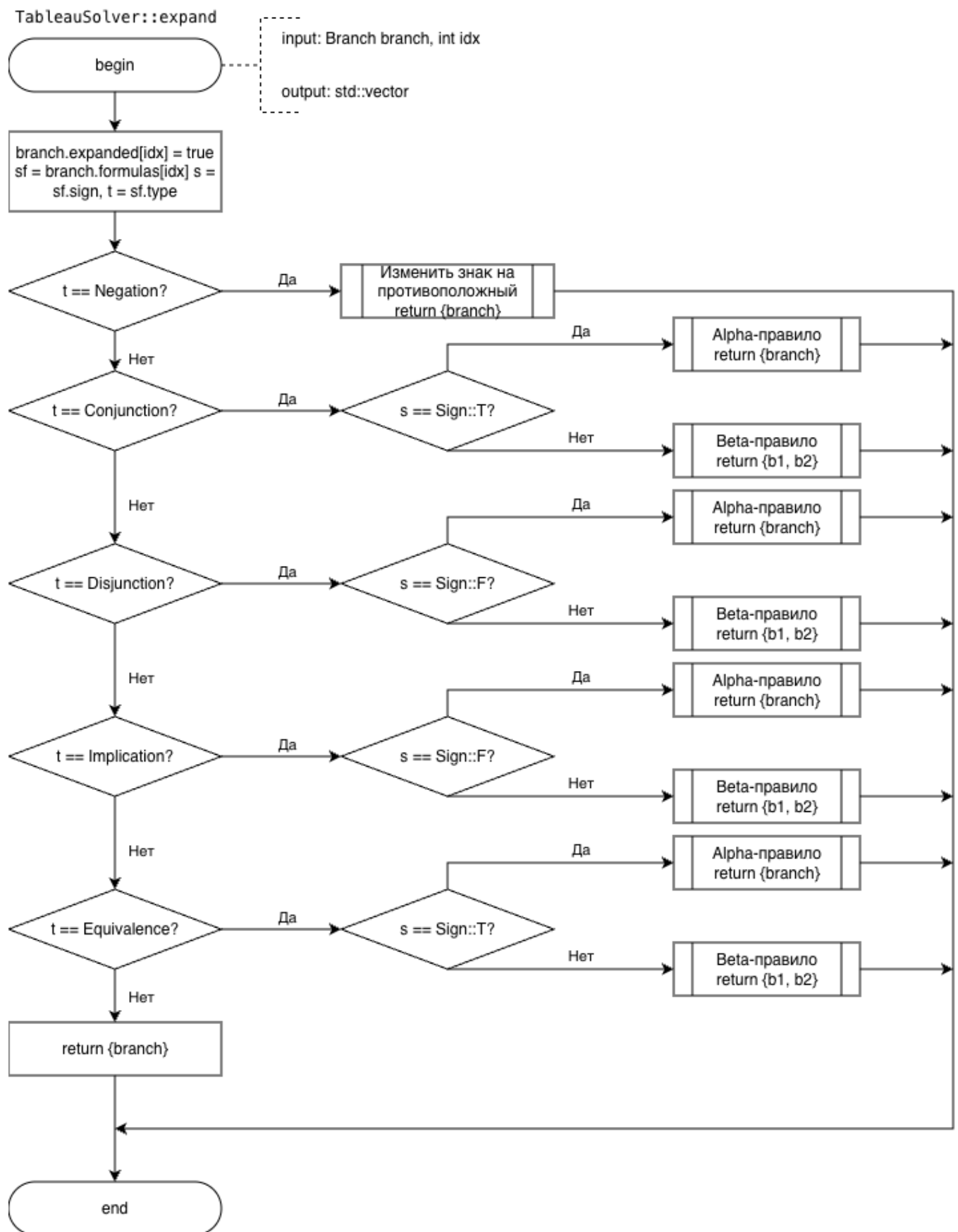


Рис 8.3. Метод expand() класса TableauSolver.

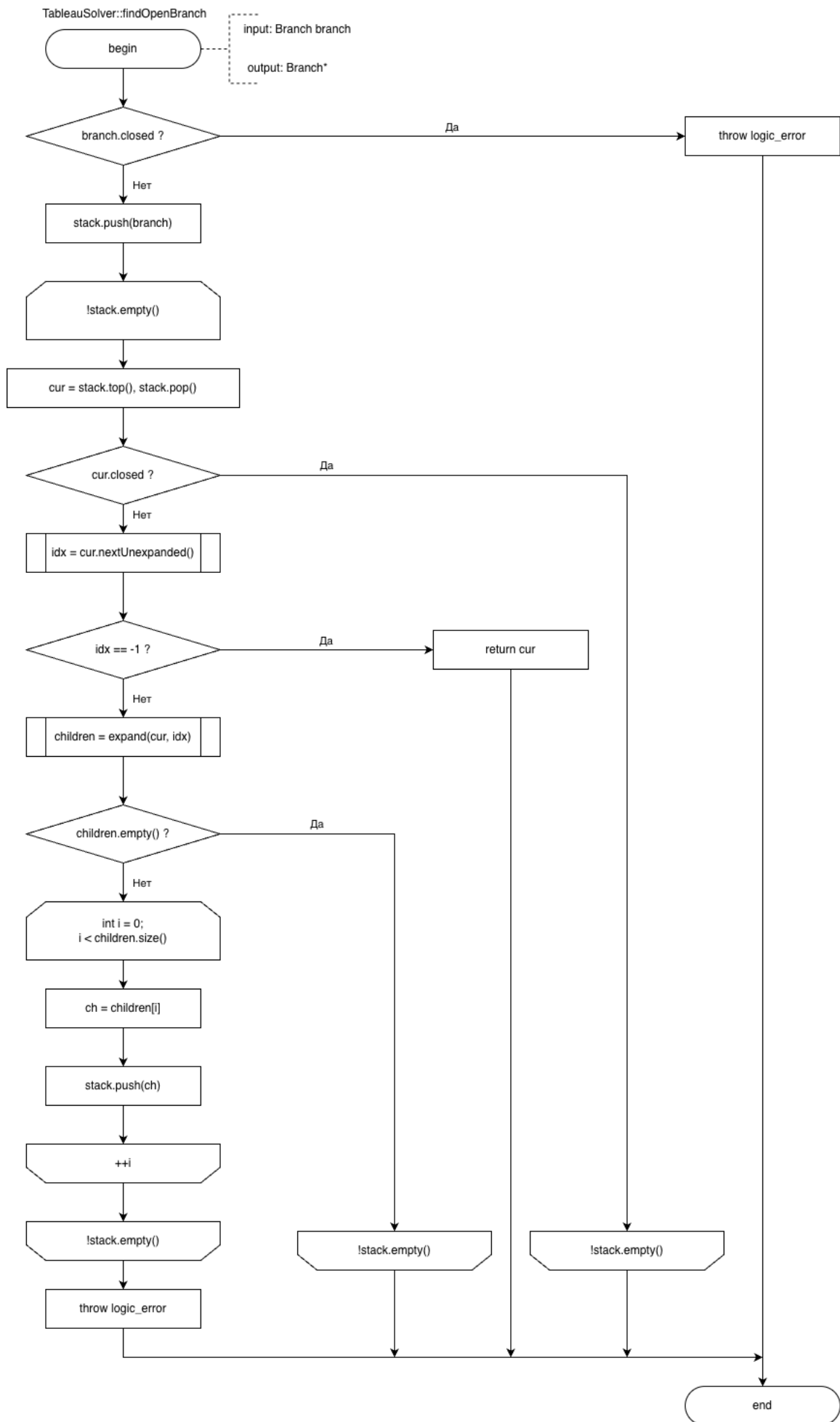


Рис 8.4. Метод findOpenBranch() класса TableauSolver.

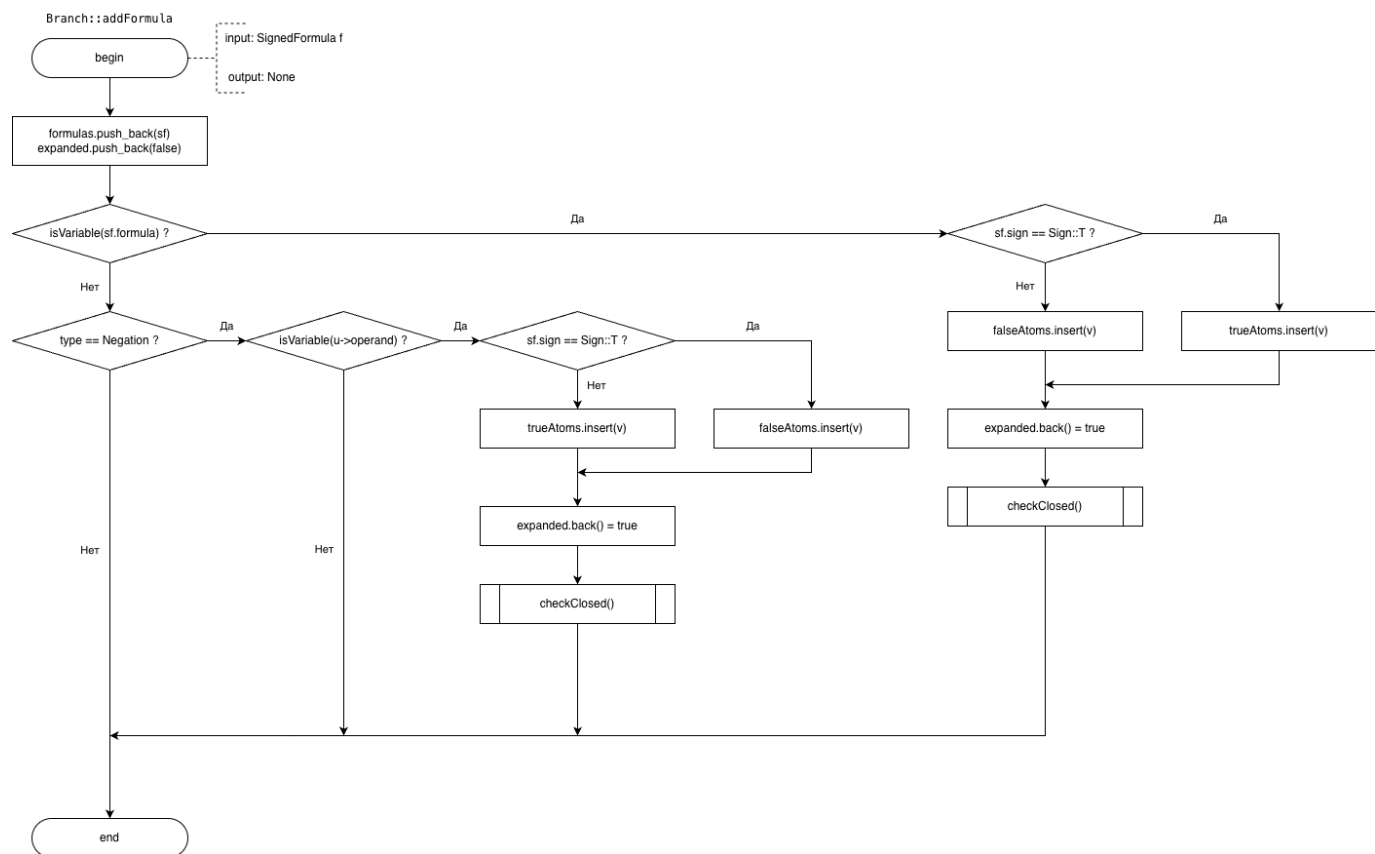


Рис 8.4. Метод addFormula() класса Branch.

Выполнение:

В качестве инструмента разработки выбран язык программирования C++. Лексический анализ входного потока реализован посредством последовательного сканирования строки с выделением лексем. Синтаксический анализ логических формул выполняется методом рекурсивного спуска, что обеспечивает строгую проверку грамматики и корректную обработку ассоциативности и приоритетов логических операций.

Внутреннее представление формулы в памяти формируется в виде абстрактного синтаксического дерева (АСТ). Для проверки логического выражения на общезначимость (тавтологию) применяется метод аналитических таблиц. Алгоритм осуществляет семантический анализ путем построения ветвей таблицы и поиска контрпримера, используя стек для обхода дерева логического вывода и правила редукции для пропозициональных связок.

При запуске приложение выводит в консоль информационный заголовок с данными студента, а также встроенную справку, описывающую правила синтаксиса для ввода логических связок, переменных и доступные системные команды. После отображения справки программа переходит в режим ожидания и запрашивает у пользователя ввод формулы для анализа.

```
=====
ЛОИС -- Проверка тавтологии методом аналитических таблиц
Студент: Аникеенко Андрей Дмитриевич, гр. 421701, БГУИР
=====

Синтаксис формул:
Переменные : латинские буквы/цифры (A, B, p, q1 ...)
!           : отрицание           пример: !A
/\          : конъюнкция          пример: A/\B
\/\         : дизъюнкция          пример: A\/B
->          : импликация          пример: A->B
~           : эквивалентность     пример: A~B
Скобки      : ( )                пример: (A/\B)->C

Команды: help | exit

Введите формулу:
```

Рис 9. Ввод формулы.

В случае необходимости пользователь может повторно вызвать встроенную справку, введя системную команду help. Программа выведет на экран актуальные правила синтаксиса логических операций, формат именования переменных и список доступных команд, после чего вернется в режим ожидания ввода:

```
Введите формулу: help

Синтаксис формул:
Переменные : латинские буквы/цифры (A, B, p, q1 ...)
!           : отрицание           пример: !A
/\          : конъюнкция          пример: A/\B
\/\         : дизъюнкция          пример: A\/B
->          : импликация          пример: A->B
~           : эквивалентность     пример: A~B
Скобки      : ( )                 пример: (A/\B)->C

Команды: help | exit

Введите формулу:
```

Рис 10. Демонстрация вызова справочной информации по команде help.

При вводе строки, содержащей символы вне заданного алфавита грамматики, лексический анализатор прерывает работу и генерирует исключение. Программа выводит на экран диагностическое сообщение с указанием конкретного недопустимого символа:

```
ОШИБКА: Лексер: недопустимый символ '\321'

Введите формулу: |
```

Рис 11. Вывод сообщения результата проверки.

При вводе корректной формулы программа выводит её канонический вид с полной расстановкой скобок и список извлеченных переменных. В данном примере результат анализа показывает, что выражение является тавтологией: программа сообщает о закрытии всех ветвей аналитической таблицы и подтверждает истинность формулы при любых значениях переменных:

```
Введите формулу: ((A -> B) /\ A) -> B

Формула      : (((A->B)/\A)->B)
Переменные   : A, B

Результат    : ТАВТОЛОГИЯ
Формула истинна при любых значениях переменных.
Все ветви аналитической таблицы закрыты.
```

Рис 12. Успешная проверка логической формулы на тавтологию.

При анализе формулы, не являющейся тождественно истинной, программа выводит её канонический вид и список извлеченных переменных. В качестве итогового результата сообщается, что выражение не является тавтологией («НЕ ТАВТОЛОГИЯ»), после чего на экран выводится найденный контрпример — конкретные истинностные значения переменных, при которых исходная формула принимает значение «ложь»:

```
Введите формулу: A \ / B -> A

Формула      : ((A\B)->A)
Переменные   : A, B

Результат    : НЕ ТАВТОЛОГИЯ
Контрпример (формула ложна при):
  A = false
  B = true
```

Рис 13. Вывод результата проверки формулы, не являющейся тавтологией.

Для корректного завершения работы приложения пользователь вводит команду quit. Программа выводит прощальное сообщение и завершает процесс выполнения с нулевым кодом возврата:

```
Введите формулу: quit
До свидания!
Program ended with exit code: 0
```

Рис 14. Завершение работы программы.

Вывод:

В ходе работы были приобретены навыки программирования алгоритмов синтаксического разбора формул языка логики высказываний.

Также была реализована процедурная программа, синтаксического разбора формул языка логики высказываний, проверяющая, является ли формула сокращенного языка логики высказываний общезначимой (тавтологией).

Теоретические сведения были взяты из следующих источников:

1. Логические основы интеллектуальных систем. Практикум: учебно-методическое пособие / В. В. Голенков, В. П. Ивашенко, Д. Г. Колб, К. А. Уваров. – Минск: БГУИР, 2011.
2. Семантическое следование и формальная выводимость (Semantic entailment and formal derivability) / Э. В. Бет // Mededelingen der Koninklijke Nederlandse Akademie van Wetenschappen. – 1955. – Т. 18, № 13. – С. 309-342.
3. Компиляторы: принципы, технологии и инструментарий / А. В. Ахо, М. С. Лам, Р. Сети, Дж. Д. Ульман. – 2-е изд. – Москва: Вильямс, 2008.